

Cuckoo Hashing with Bounded Displacement for Exact-Match Flow Tables

Soumyajit Kolay

School of Computer Engineering

KIIT Deemed University

Bhubaneswar, India

2650026@kiit.ac.in

Abstract—THE ABSTRACT OF THIS DOCUMENT IS YET TO BE UPDATED. I am learning L^AT_EX as a part of this research.

Index Terms—SDN, hashtables, d-ary cuckoo, bloom filter, flow tables

I. INTRODUCTION

The dictionary abstraction—supporting insertion, deletion, and membership queries over a dynamic key set—sits at the core of nearly every high-throughput networking data plane, and nowhere is its performance more scrutinized than in the exact-match flow table of a software-defined switch. Every packet traversing the data plane must be matched against this table at line rate, so a single slow lookup anywhere in the tail of the latency distribution translates directly into queuing delay or packet loss. Classical collision-resolution schemes such as chained hashing and open addressing with linear probing offer excellent average-case behavior but degrade to logarithmic or, in adversarial cases, linear worst-case lookup cost as the table fills—a property that is simply incompatible with a hardware or software pipeline built around a fixed per-packet time budget.

Cuckoo hashing was introduced by Pagh and Rodler as a direct answer to this problem [1]. By allowing each key two candidate locations across two tables and permitting keys already resident in the table to be displaced (“kicked”) to their alternate location on collision, the scheme guarantees exactly two probes for every lookup—worst case, not merely expected. This property alone made cuckoo hashing an immediate candidate for line-rate dictionaries, and it has since been adopted widely in networking contexts precisely for this reason. Yet Pagh and Rodler’s original construction carries a structural cost that later work has spent almost two decades trying to manage: when a chain of displacements returns to a previously visited cell, the insertion procedure has entered a cycle, and the only recourse in the original scheme is to abandon the current hash functions entirely and rehash the full table—an $O(n)$ operation that, however rare, is catastrophic if it occurs during a latency-sensitive insertion at line rate.

The subsequent literature can be read as a sustained effort to defer, shrink, or altogether avoid this full-table rehash. Kirsch, Mitzenmacher, and Wieder showed that reserving a small constant-sized side table—a stash—for the

rare items that fail to settle after a bounded number of displacements reduces the probability of ever needing a full rehash from polynomial to a much smaller $\Theta(n^{s-1})$ for a stash of size s , with a stash of only three or four entries already yielding a dramatic practical improvement [Kirsch2009stash]. Kutzelnigg subsequently tightened this analysis, showing that the same failure-probability bounds hold even without the extra verification step Kirsch et al.’s original proof required, simplifying the practical insertion routine [Kutzelnigg2010stash]. Aumüller, Dietzfelbinger, and Woelfel then relaxed the hash-function assumptions underlying the stash guarantee, proving that simple, explicit 2-independent hash families suffice to preserve the $O(n^{s-1})$ failure bound and $O(s)$ query time, removing the reliance on fully random hash functions that had limited the scheme’s practicality [aumuller2014explicit]. On the load-capacity side, Fountoulakis and Panagiotou resolved a long-open question by deriving the exact load thresholds for d -ary cuckoo hashing with $d \geq 3$ choices per key—for instance, showing that three choices already permit roughly 92% table utilization—giving practitioners, for the first time, precise rather than merely empirical guidance on how sparse a d -ary table must be kept [fountoulakis2012sharp].

A parallel line of work adapted cuckoo hashing specifically to the networking substrate rather than treating it as a general-purpose dictionary. Pontarelli et al. proposed a parallel d -pipeline implementation targeting ASIC and FPGA memories accessed concurrently, reporting throughput gains of up to 60% over prior parallel cuckoo designs for exact-match lookup [pontarelli2016parallel], and later introduced Cuckoo Cache, which exploits the skewed popularity distribution of real network flows by biasing frequently accessed items toward the first-probed table, yielding substantial query-throughput improvements on real traffic traces without altering the underlying cuckoo guarantees [pontarelli2016cuckoocache]. Le Scouarnec pushed cache-efficiency further on commodity CPUs with Cuckoo++, an implementation tailored to Intel Xeon cache-line behavior that outperformed the widely used DPDK hash table by 45–70% on lookups while leaving the classical theoretical analysis of cuckoo hashing untouched [lescouarnec2018cuckoopp]. Minaud and Papanthou revisited and corrected a subtle error in the

original multi-item-per-bucket stash analysis of Kirsch et al., establishing the true asymptotic failure probability as $\Theta(n^{-d-s})$ rather than the previously claimed bound—[Minaud2023generalized](#)—a correction of direct consequence to any system, like the one proposed here, that relies on stash-failure guarantees for its worst-case claims. Noble subsequently closed a longstanding gap in the theory by deriving the first explicit, closed-form failure-probability bounds for cuckoo hashing with a stash of arbitrary—not merely constant—size, removing the asymptotic hedging present in all prior stash analyses—[Noble2021explicit](#).

Most recently, Kuzmaul and Mitzenmacher introduced bubble-up cuckoo hashing, a d -ary insertion policy that achieves an essentially optimal number of hash locations while guaranteeing $O(\delta^{-1})$ expected insertion time at any load factor $1-\delta$ and $O(1)$ expected positive-query time independent of d —the strongest theoretical insertion-time guarantee for d -ary cuckoo hashing to date—[Kuzmaul2025bubbling](#). On the hardware side, Megías et al. designed an FPGA implementation that exploits parallel insertion iterations to cut the number of reallocation rounds by more than 60% at loads beyond 95%, targeting exactly the real-time communications-network setting this work is concerned with—[Megias2025minimizing](#). Closest of all to the present application, Xiong et al. proposed TECache, a nearly conflict-free hashing scheme for TCAM-backed SDN flow tables that uses three candidate locations with co-directional kicking to sustain high cache-hit rates and substantial energy savings under real backbone traffic—[Xiong2025tecache](#)—a system that, notably, optimizes for hit rate and power rather than for a bound on worst-case insertion cost. Tripathi et al. most recently proposed a new set of performance indicators for sequential cuckoo hashing variants, aimed at giving practitioners richer evaluation vocabulary than raw throughput alone—[Tripathi2026performance](#).

The stash-based line of theoretical work—[Kirsch2009stash](#), [Kutznigg2010stash](#), [Aumuller2014explicit](#), [Minaud2023generalized](#), [Noble2021explicit](#)—establishes precisely *how likely* a full rehash is for a given stash size, but every one of these analyses assumes the classical insertion procedure: displacement continues, essentially without an explicit external bound, until either an empty slot is found or a cycle is detected, at which point the stash is invoked. None of them treat the length of the eviction chain itself as a tunable, deliberately constrained parameter—the stash is a safety net for whatever the unbounded process eventually fails to place, not a fallback triggered *early* by design. Conversely, the networking-systems line of work—[Pontarelli2016parallel](#), [Pontarelli2016cuckoocache](#), [Lescouarnec2018cuckoop](#), [Xiong2025tecache](#)—optimizes throughput, cache efficiency, or energy consumption on real traffic, but treats worst-case insertion cost as a secondary concern; TECache in particular reports hit rate and power savings but offers no guarantee on how long a single insertion can take

before either succeeding or falling back to a full rehash. The most recent theoretical advance, bubble-up cuckoo hashing—[Kuzmaul2025bubbling](#), achieves a strong *expected* insertion-time bound at high load, but it is an offline-insertion-time result validated against random-graph theory, not an online scheme instrumented against the specific metrics—worst-case per-insertion latency, stash utilization, and rehash frequency under a synthetic or real flow-table workload—that a line-rate SDN deployment must be evaluated on.

The gap, concretely, is this: no existing construction couples an *explicitly and tightly bounded* kick-chain length with *early* stash fallback—triggering the stash before the classical process would otherwise continue searching—while confining rehashing to the rare event of stash overflow rather than ordinary cycle detection, and then characterizes the resulting system on the metrics that matter for exact-match flow tables specifically: worst-case and average lookup latency, insertion success rate, stash utilization, and memory overhead, benchmarked against the standard baselines of chained hashing, unstashed 2-way cuckoo hashing, and linear probing on real or synthetic microflow traces.

This paper closes that gap by treating the kick-chain bound itself as the primary design variable rather than an afterthought. The proposed construction combines d -ary candidate slot selection—[Fountoulakis2012sharp](#), [Kuzmaul2025bubbling](#)—with a small Kirsch-style stash—[Kirsch2009stash](#)—whose sizing is informed by Noble’s tightened, general-stash bounds—[Noble2021explicit](#)—and corrected against the Minaud–Papamanthou erratum—[Minaud2023generalized](#), but departs from all of these in one specific respect: the eviction procedure is capped at a deliberately short `MaxKicks` bound, well below what a cycle-detection-only policy would tolerate, so that displaced keys are routed to the stash *proactively* rather than only after the classical process has exhausted its search. Rehashing is triggered exclusively by stash overflow and is performed incrementally—reinserting only the stash’s contents—rather than as a full-table rebuild, directly targeting the catastrophic-cost scenario that has motivated every stash-based scheme since Kirsch et al.—[Kirsch2009stash](#)—without adopting their assumption of an effectively unbounded search process beforehand. A load-factor monitor, sized using the exact thresholds established by Fountoulakis and Panagiotou—[Fountoulakis2012sharp](#), resizes the table proactively rather than reactively, keeping the stash-overflow event itself rare by construction.

The resulting scheme is positioned deliberately alongside, rather than in competition with, the closest systems-level precedent: TECache—[Xiong2025tecache](#)—demonstrates that a cuckoo-style cache can sustain high hit rates and energy savings for SDN TCAM flow tables, but offers no bounded worst-case insertion guarantee; this work supplies exactly that guarantee, evaluated using the same class of real and synthetic exact-match traffic that TECache and Cuckoo Cache—[Pontarelli2016cuckoocache](#)—use to

validate throughput, but reoriented around the central empirical question a bounded-displacement design raises: how short can the kick-chain bound be made before the resulting rise in stash utilization and rehash frequency becomes unacceptable, at a given table load factor. Answering that question, with the explicit theoretical grounding provided by~\cite{noble2021explicit} and the corrected bounds of~\cite{minaud2023generalized}, is the specific contribution this paper makes to the cuckoo hashing literature surveyed above.

II. REFERENCES

REFERENCES

- [1] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.